What Should a Local Agent Remember?

Saving state for agents running at the edge · Eoin Jordan · one page to take back on Monday

Model for the workload
Start with the questions the agent asks when things go wrong. The document's shape follows the questions, not the data.

Index for the questions
Equality, then sort, then range. Every index is a bet on how the system is used, and every write pays for it.

Treat AI memory as data
Persist what matters, expire what doesn't, and make every action explainable from a record.

1 · THE QUESTIONS, BEFORE THE SCHEMA

- What runs are active for this project?
- What changed since the last good run?
- Which jobs are still pending?
- Which actions were approved by a human?
- What previous failure looks like this one?

2 · ONE RUN = ONE DOCUMENT

```
{ runId: "run-1842", projectId: "project-512482", orgId: "org-71",
  objective: { metric: "accuracy", target: 0.92 },
  status: "training",
  configuration: { windowMs: 2000, learningRate: 0.001 },
  jobs: [ { jobId: "job-203", type: "training", status: "running" } ],
  results: { accuracy: 0.89, latencyMs: 14 },
  approvals: [ { action: "train", by: "human", at: ... } ],
  createdAt: ..., updatedAt: ... }
```

What grows without bound? What changes every few seconds? What needs a history, not a field? Embed what is small, bounded and read with the parent (config snapshot, latest result, current status). Reference what is large, unbounded or shared (job events, training logs, artifacts). Append-only log for the explanation trail (actions, status transitions, approvals).

3 · THE QUESTION CREATES THE INDEX

```
runs.find({ projectId: "project-512482", status: "training" })
  → { projectId: 1, status: 1 }
runs.find({ projectId: "project-512482" }).sort({ updatedAt: -1 }).limit(1)
  → { projectId: 1, updatedAt: -1 }
```

Check with `explain("executionStats")`: keys examined should equal documents returned. The trade: every status update now maintains both indexes.

4 · FROM POLLING TO EVENTS

```
pipeline = [{"$match": {"operationType": "update",
  "updateDescription.updatedFields.status": {"$in": ["completed","fai
with runs.watch(pipeline, full_document="updateLookup",
                resume_after=saved_token) as cs:
    for change in cs:
        run = change["fullDocument"]
        agent.resume(thread_id=run["runId"])
        save(change["_id"])          # the resume token
```

The job updates its own document; the database rings the doorbell; the consumer resumes the agent. Design the failure path: resume token for downtime, idempotent handler for duplicates, a scan by `updatedAt` pas oplog window.

5 · MEMORY HAS A HALF-LIFE

```
db.sessions.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 0 })
// the writer stamps expiresAt per document, at write time
```

Short-term (temp job state, session context, cache, tool outputs): expir **term** (audit history, summaries, approvals, baselines): keep, index, audit and failed attempts: review, then decide. TTL sweeps about once a minu never applies to audit.

6 · AN AGENT THAT CAN ACT MUST BE EXPLAINABLE

```
{ action: "start_training", requestedBy: "agent", approvedBy: "eoin",
  scope: { orgId: "org-71", projectId: "project-512482" },
  decision: "approved", policyId: "train-non-destructive",
  reason: "non-destructive training run", at: ISODate(...) }
```

A schema refuses actions outside the vocabulary at insert. A governor r policy collection before approving. Every decision lands in an audit coll The approval record is also the lock: one conditional update from appr executing wins.

MONDAY

- **Developer:** give every workflow a document and a thread id; replace polling loop with a change stream.
- **Architect:** put a state layer between tools and the world; decide emb reference by read pattern.
- **Platform lead:** index from the query log; make lifetimes a schema de keep audit and approvals out of the expiry list.
- **Manager:** ask one question of any agent project: can it explain what the answer is not a collection, it is not ready.

CC BY 4.0